Assignment-Classification Algorithm

Problem Statement or Requirement:

As a Data Scientist, I could identify certain solutions for the problem:

1. Problem Statement – Predict the Chronic Kidney Disease(CKD)
2. Problem Identification:
   Stage 1: Machine Learning
   Stage2: Supervised Learning
   Stage3: Classification
3. Dataset: 1. Total no of rows:399
   2. Total no of columns: 25
   3. Output is classification (Last column)
4. Pre Processing Method: This is a Multi Linear classification / Sg,rbc,pc,pcc,ba,htn,dm,cad,appet,pe,ane,classification(Coloumns) to be changed to Categorized Data as Python takes only 1 & 0 & also implementing grid search method.

5. SVM

```
print("The confusion Matrix:\n",cm)
```

```
The confusion Matrix:
 [[51  0]
 [ 1 81]]
```

```
print("The report:\n",clf_report)
```

```
The report:
               precision    recall  f1-score   support

           0       0.98      1.00      0.99        51
           1       1.00      0.99      0.99        82

    accuracy                           0.99       133
   macro avg       0.99      0.99      0.99       133
weighted avg       0.99      0.99      0.99       133
```

```
from sklearn.metrics import roc_auc_score

roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])
```

```
1.0
```

6. Decision Tree

```
print("The confusion Matrix:\n",cm)
```

```
The confusion Matrix:
 [[51  0]
 [ 3 79]]
```

```
print("The report:\n",clf_report)
```

```
The report:
               precision    recall  f1-score   support

           0       0.94      1.00      0.97        51
           1       1.00      0.96      0.98        82

    accuracy                           0.98       133
   macro avg       0.97      0.98      0.98       133
weighted avg       0.98      0.98      0.98       133
```

```
from sklearn.metrics import roc_auc_score

roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])
```

```
0.9817073170731707
```

7. Random Forest

```
print("The confusion Matrix:\n",cm)
```

```
The confusion Matrix:
 [[50  1]
 [ 0 82]]
```

```
print("The report:\n",clf_report)
```

```
The report:
               precision    recall  f1-score   support

           0       1.00      0.98      0.99        51
           1       0.99      1.00      0.99        82

    accuracy                           0.99       133
   macro avg       0.99      0.99      0.99       133
weighted avg       0.99      0.99      0.99       133
```

```
from sklearn.metrics import roc_auc_score

roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])
```

```
0.9998804399808704
```

8. Logistics

```
print("The confusion Matrix:\n",cm)
```

```
The confusion Matrix:
 [[51  0]
 [ 1 81]]
```

```
print("The report:\n",clf_report)
```

```
The report:
               precision    recall  f1-score   support

           0       0.98      1.00      0.99        51
           1       1.00      0.99      0.99        82

    accuracy                           0.99       133
   macro avg       0.99      0.99      0.99       133
weighted avg       0.99      0.99      0.99       133
```

```
from sklearn.metrics import roc_auc_score
roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])
```

```
1.0
```

9. K-Nearest Neighbor

```
print("The confusion Matrix:\n",cm)
```

```
The confusion Matrix:
 [[51  0]
 [ 4 78]]
```

```
print("The report:\n",clf_report)
```

```
The report:
               precision    recall  f1-score   support

           0       0.93      1.00      0.96        51
           1       1.00      0.95      0.97        82

    accuracy                           0.97       133
   macro avg       0.96      0.98      0.97       133
weighted avg       0.97      0.97      0.97       133
```

```
from sklearn.metrics import roc_auc_score
roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])
```

```
0.9873266379722621
```

10. Naïve Bayes

```
The f1_macro value for best parameter {'var_smoothing': 1e-08}: 0.9775556904684072
```

```
7]: print("The confusion Matrix:\n",cm)
```

```
The confusion Matrix:
 [[51  0]
 [ 3 79]]
```

```
8]: print("The report:\n",clf_report)
```

```
The report:
               precision    recall  f1-score   support

           0       0.94      1.00      0.97        51
           1       1.00      0.96      0.98        82

    accuracy                           0.98       133
   macro avg       0.97      0.98      0.98       133
weighted avg       0.98      0.98      0.98       133
```

```
9]: from sklearn.metrics import roc_auc_score
    roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])
```

```
9]: 1.0
```

Conclusion:

According to me, SVM method gave a better result & I would like to deploy this method for better r value.